

MovieLens Project

Henrik B. Sørensen

2026-06-03

Contents

Introduction	2
Project Goal	2
Disclaimer:	2
Data set introduction	3
movies.dat	3
ratings.dat	3
tags.dat	3
Environment Setup	4
Method	5

Introduction

This paper is the MovieLens Project for the Data Science: Capstone, HarvardX PH125.9x. A course in Machine Learning based on the R programming language.

In this project, I will analyze a (limited) dataset for movie ratings and attempt to develop a model for predicting the rating of specific movies using a limited training set. Having created a model, I will test it using the test dataset. Overall, the steps performed can be summarized into:

- Setup the environment
 - Install necessary libraries (if missing)
 - Include necessary libraries
- Prepare the dataset for initial analysis
 - Download dataset (if missing) and unpack
 - Split the dataset into training set/test set
- Analyze and filter the dataset
 - Visualize data datasets to identify “noise”
 - Filter out outliers to remove “noise”
 - Visualize the data trends to come up with a plan on how to develop the model
- Model Development
- Model Testing and Verification
- Model Extension and Testing/Verification
- Model Analysis
- Conclusion

Project Goal

The goal of this project is to come up with a come up with a model with an RMSE (Root Mean Square Error) < 0.86490 (or as low as possible) for predicting movie ratings based on genres (and analyze if the model can be optimized when taking tags and timestamps into account:

Could there be a correlation between:

- rating and genre?
- rating and month/time of year?
- rating and tags?

Disclaimer:

The R code in this project is not optimized for speed and/or memory layout in any way. It is written for functionality and readability. Also, the code will attempt to use the “download if missing” and “install if missing” approach to both datasets and libraries.

Data set introduction

Before moving on to the actual analysis and R programming, let's take a look at the actual data itself: Click [here](#) to download the raw dataset. When uncompressed, the dataset consists of 3 files (when ignoring the Readme.html description file and support files): movies.dat, ratings.dat and tags.dat (though, not used in the example). All files are in the csv format delimited by ::.

movies.dat

This file holds a “database” of movies, which have been set up for rating in the form of a text file: An example line of this is “1::Toy Story (1995)::Adventure|Animation|Children|Comedy|Fantasy”.

Which is to be read in the format of:

movieId	title	genres
1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy

The genres column needs to be split into several columns and loaded into another table for further analysis. This table/dataset holds 9000055 rows.

ratings.dat

This file holds a “database” of ratings, which have been given for the listed movies in the form of a text file: An example line of this is “1::122::5::838985046”.

Which is to be read in the format of:

userId	movieId	rating	Timestamp
1	122	5	1996-08-02 11:24:06

There's a hard one directional coupling between the ratings (movieId) and movies (movieId). A movie must be present for it to be rated.

This table/dataset holds 10000054 rows.

tags.dat

This file holds a “database” of tags connected to the movies by users in the form of a text file: An example line of this is “15::4973::excellent!::1215184630”.

Which is to be read in the format of:

userId	movieId	Tag	Timestamp
15	4973	excellent!	2008-07-04 15:17:10

Dimmer

There's a hard one directional coupling between the tags (movieId, userId), the rating (movieId, userId) and movies (movieId). A movie must be present for it to be rated.

This table/dataset holds 10000054 rows.

Please note, that the ratings table data is not mandatory for the movie ratings and was not included in the example code provided. However, we'll explore, if there's a connection between tags and ratings later.

Environment Setup

The project was done entirely in RStudio 2026.05.0+218 “Golden Wattle” for windows with corresponding R version 4.5.2 on a Windows 11 Home Edition. Required tinytex was also installed following instructions provided by RStudio. The project makes use of several R packages, which are required for the R code to run. Keeping in the spirit of the “install if missing” philosophy, we’ll make use of the following syntax in order to test for the existence of a library and install it if missing:

```
if(!require(<library name>))  
  install.packages("<library name>", repos = "http://cran.us.r-project.org")
```

The require function will return FALSE, if the desired library has not been installed. For example, installing tidyverse would look like:

```
if(!require(tidyverse))  
  install.packages("tidyverse", repos = "http://cran.us.r-project.org")
```

Method